
Scheduling Toolkit

Release 0.1.0

Scheduling Toolkit contributors

Oct 06, 2026

DOCUMENTATION

1	Getting started	3
1.1	Optional capabilities	3
2	Code Documentation	5
2.1	Tasks Module	5
2.2	Visualization Module	6
2.3	Scheduling Models	6
2.4	synthdata Module	7
3	Use cases	13
3.1	PCB call-center workload planning	13
4	Indices and tables	35
	Python Module Index	37
	Index	39

Scheduling Toolkit is a Python library for task modeling, resource assignment, and scheduling strategies. This documentation starts with the general-purpose library. A historical PCB workload-planning application is presented separately as an optional use case.

GETTING STARTED

Install the core library with:

```
python -m pip install scheduling-toolkit
```

The core task model and FCFS heuristic need no third-party runtime dependencies:

```
from scheduling import FCFSP, Tasks

tasks = Tasks()
tasks.add_task("prepare", release=0, due=10, process={"worker": 3})
tasks.add_task("review", release=1, due=12, process={"worker": 2})
schedule = FCFSP(tasks)()
```

FCFSP is a heuristic and does not enforce due dates or prerequisites. Use `MultiMachineMILP` or `MultiMachineQP` when optimization or prerequisite constraints are required. Those models need the `optimization` extra and a separately installed solver executable:

```
python -m pip install "scheduling-toolkit[optimization]"
```

```
from scheduling import MultiMachineMILP

result = MultiMachineMILP(tasks, solver="glpk", respect_due=True)()
```

Pass solver-specific settings with `solver_options={...}`; pass `executable="/path/to/solver"` when the solver is not discoverable on `PATH`. The result dictionary contains a "Message" status entry.

1.1 Optional capabilities

- `simulation` adds NumPy, SciPy, and joblib for stochastic evaluation.
- `visualization` adds Matplotlib for Gantt charts.
- `data` adds NumPy, SciPy, and pandas for synthetic data tools.

Each feature can be installed independently, or combine extras as needed. Domain-specific examples are documented separately under *Use cases*.

CODE DOCUMENTATION

2.1 Tasks Module

class `scheduling.Task.Tasks`

Initiates an object to add tasks to a tentative task list.

The following attributes are associated to a task:

- Task name: a string place holder for the task
- Release date: the earliest time that a task can be processed
- Due date: a date-time that the task should be completed by then
- Process time: the length of time that a resources requires to finish a given task
- Start time: a specific date-time for the start of a task
- Unavailable: a period where a resources is not available (takes time off, etc.)
- Prerequisite: when task A should be finished before task B

A *Tasks* instance handles the association of the above attributes in a task list through 3 methods.

add_prerequisite(*task1*, *task2*)

Sets prerequisite for tasks

Parameters

- **task1** – the task to be followed by *task2*
- **task2** – the followup task

Returns

None

add_task(*task_name=None*, *release=0*, *due=1*, *weight=1*, *process=None*, *start=None*)

Adds a task to the list to be scheduled

Parameters

- **task_name** – *optional*, a given name for the task
- **release** – Release date; the earliest time that a task can be processed
- **due** – Due date; a date-time that the task should be completed by then
- **weight** – *optional*, contribution coefficient to the cost/profit
- **process** – a dictionary of the task's processing time for each resources, capable of handling the task

- **start** – *optional*, a specific date-time for the start of a task

Returns

None

schedule_unavailable(*resource, process, release=0, start=None, due=None*)

Schedules an unavailable time for a resources

Parameters

- **resource** – the name of the resources
- **process** – the length of time off
- **release** – the earliest time to be unavailable
- **start** – a sharp date-time to take time off
- **due** – the time resources returns (becomes available)

Returns

None

2.2 Visualization Module

class scheduling.Visuals.**Visuals**(*schedule*)

A class to visualize an schedule. An instance of *Visuals* accepts one argument:

Parameters

schedule – An instance of *Schedule*.

gantt(***kwargs*)

Plots a gantt chart of the given schedule

Returns

the *matplotlib* plot object

2.3 Scheduling Models

class scheduling.MultiMachine.**MultiMachineMILP**(*tasks=None, **kwargs*)

This is the class implementing the classical MILP version of Machine-Task scheduling. The class takes advantage from Pyomo which allows python code to call most well-known optimizers like CPLES, Gurobi, Knitro, etc.

Parameters

- **tasks** – a loaded *Tasks* instance
- **kwargs** – a dictionary with the following optional values:
 - **solver**: the optimizer engine. Default is set to the open source *glpk*
 - **executable**: The path to the engines executable, if not available under *PATH*
 - **respect_due**: Boolean, default is False. If True, forces model to find an optimal schedule respecting all due dates
 - **cost**: Boolean, default is False. If True, the objective is set to minimize cost
 - **priority**: Boolean, default is False. If True, the objective is set to minimize weighted delay

2.4 synthdata Module

This module provides basic framework for generating synthetic data resembling an existing dataset. One could determine types of each field and the possible values for each field. Then the `InspectData` class will produce data based on the given types. Moreover, one can associate marginal distributions to each field or a joint distribution for data generation. If no distribution is associated, then the data will be generated uniformly over the required ranges.

2.4.1 Supported data types

The following data types are supported:

- ***SynthBin***
Support for binary data, i.e., the data fields consisting of 0, 1 values;
- ***SynthInt***
Support for integer valued data;
- ***SynthReal***
Support for real valued data;
- ***SynthCat***
Support for categorical type of data, i.e., the discrete variables whose values are predetermined;
- ***SynthDate***
Support for datetime data;

Each of these data types accept *data* that is a 1-d `numpy.array` and *rv* which is a `scipy.stats` distribution, implementing `rv.rvs` to generate samples. Among the above, `SynthInt` and `SynthReal` accept two parameters *a* and *b* which are the lower and upper bounds of the sampling interval respectively. `SynthDate` accepts *fmt* which determines the date formatting for the input data.

2.4.2 Generating Synthetic Data

The `SynthData` class is responsible for generating synthetic data based on types, distributions and relations defined on the data. One initiates an instance as:

```
sd = SynthData(df, default_rv='uniform', distribution_type='marginal', rv=None)
```

where *df* is the pandas dataframe that will be synthesized. The rest of arguments are optional:

- **`default_rv`**
determines the default distribution for those fields where no distribution is associated to. If the `distribution_type` is set to 'joint' this will be ignored.
- **`distribution_type`**
determines if the distribution(s) calculated based on *df* are *marginals* or a single *joint* distribution.
- **`rv`**
determines a predefined distribution for joint distribution.

To set the type of a column of *df* one should use `set_type` method. This method accepts a list of column names, their types and a tuple of initiating parameters. Every column in the `columns` list will be given the same type. The type could be either an instance of `SynthBin`, `SynthInt`, `SynthReal`, `SynthCat`, `SynthDate`, or a string determining the type, e.g., 'bin', 'int', 'real', 'cat', 'date'. If no type is associated to a column, it is assumed to be of categorical type.

The final command which generates the synthetic data is `sample(num)` where 'num' is the number of synthetic samples to be generated. This method will return a `pandas.DataFrame` containing synthetic data of size 'num'.

2.4.3 Constraints

It is quite common that the values of some fields in a record depend on other fields. Simple constraints on the values of a field and relations between pairs can be handled using `field` objects.

If it is required to impose a constraint on a column, one can use `where()` method. The statement to add a constraint would generally look like `sd.where(field('clmn') > val)` or `sd.where(field(clmn1) <= field(clmn2))`. The acceptable operators are `==`, `!=`, `>`, `<`, `>=`, `<=`, `in`, `nin`. The operators `in` and `nin` check membership of elements of 'clmn' in 'val' which has to be an iterable or membership in the column 'clmn2'. The `in` stands for belonging and `nin` stands for *not in*.

2.4.4 Code documentation

class `scheduling.synthdat.SynthBase(data=None, rv=None)`

The base class for various synthetic data types.

static `get_val(x)`

Converts the value of *x* into numeric that can be handled by random distributions

Parameters

x – the value to be converted into numeric

Returns

the corresponding numeric value

ret_val(X)

Similar to `set_val` but works on the data stored in *X*

Parameters

X – the data to be converted to numbers

Returns

translation of *X*

set_val()

Generates the numeric translation of the given data.

Returns

list of values

class `scheduling.synthdat.SynthBin(data=None, rv=None)`

Support for binary data, i.e., the data fields consisting of 0, 1 values

get_val(x)

Converts the value of *x* into numeric that can be handled by random distributions

Parameters

x – the value to be converted into numeric

Returns

the corresponding numeric value

ret_val(X)

Similar to `set_val` but works on the data stored in *X*

Parameters

X – the data to be converted to numbers

Returns

translation of *X*

set_val()

Generates the numeric translation of the given data.

Returns

list of values

class scheduling.synthdat.**SynthCat**(*data*, *rv=None*)

Support for categorical type of data, i.e., the discrete variables whose values are predetermined;

get_val(*x*)

Coverts the value of *x* into numeric that can be handled by random distributions

Parameters

x – the value to be converted into numeric

Returns

the corresponding numeric value

ret_val(*X*)

Similar to *set_val* but works on the data stored in *X*

Parameters

X – the data to be converted to numbers

Returns

translation of *X*

set_val()

Generates the numeric translation of the given data.

Returns

list of values

class scheduling.synthdat.**SynthData**(*df*, *default_rv='uniform'*, *distribution_type='marginal'*, *rv=None*)

A class which takes a *real pandas.DataFrame* and produces *synthetic* data similar to the real data based on types and distributions provided by the user and/or extracted out of original data.

Parameters

- **df** – a *pandas.DataFrame* containing original data.
- **default_rv** – default distribution of columns; default ‘uniform’. Also could be ‘normal’. Only effective if *distribution_type* is ‘marginal’, otherwise will be ignored.
- **distribution_type** – default ‘marginal’. Determines the type of distribution. If ‘joint’, then either a normal distribution is calculated based on provided data or will use *rv* if *rv* is not ‘None’.
- **rv** – default ‘None’. The joint distribution of variables. Only effective if *distribution_type* is ‘joint’.

filter(*df*)

Filter the ‘df’ to remove illegal records according to constraints

Parameters

df – the dataframe to be filtered

Returns

the filtered dataframe

generate(*num*)

internal Generate *num* synthetic data records without considering constraints

Parameters

num – number of samples

Returns

`pandas.DataFrame`

sample(*num*)

Produces ‘num’ records of synthetic data following given types, distributions and constraints

Parameters

num – number of synthetic data records

Returns

a dataframe consisting of ‘num’ synthetic records.

set_type(*clmns*, *typ*, *param=None*)

Define the type of columns.

Parameters

- **clmns** – a *list* of df columns
- **typ** – the associated type, either a string (‘bin’, ‘int’, ‘real’, ‘cat’, ‘date’) or an instance of `SynthBin`, `SynthInt`, `SynthReal`, `SynthCat`, `SynthDate`.
- **param** – parameters to be passed to synthetic data type if a string is given for ‘*typ*’. It could be a couple (a, b) for ‘int’ and ‘real’ type and just the format for ‘date’.

transform()

internal to analyse and initialize data types and convert them to numerical values.

Returns

`None`

where(*cns*)

Add a constraint of values of a column using `field` objects.

Parameters

cns – the constraint like `field(clmn1) > val1` or `field(clmn1) <= field(clmn2)`.

Returns

`None`

class `scheduling.synthdat.SynthDate`(*data*, *frmt='%Y-%m-%d'*, *rv=None*)

Support for datetime data;

get_val(*x*)

Coverts the value of *x* into numeric that can be handled by random distributions

Parameters

x – the value to be converted into numeric

Returns

the corresponding numeric value

ret_val(*X*)

Similar to `set_val` but works on the data stored in *X*

Parameters

X – the data to be converted to numbers

Returnstranslation of X **set_val()**

Generates the numeric translation of the given data.

Returns

list of values

class scheduling.synthdat.**SynthInt**(*a=None, b=None, data=None, rv=None*)

Support for integer valued data;

get_val(x)Coverts the value of x into numeric that can be handled by random distributions**Parameters****x** – the value to be converted into numeric**Returns**

the corresponding numeric value

ret_val(X)Similar to *set_val* but works on the data stored in X **Parameters****X** – the data to be converted to numbers**Returns**translation of X **set_val()**

Generates the numeric translation of the given data.

Returns

list of values

class scheduling.synthdat.**SynthReal**(*a=None, b=None, data=None, rv=None*)

Support for real valued data;

get_val(x)Coverts the value of x into numeric that can be handled by random distributions**Parameters****x** – the value to be converted into numeric**Returns**

the corresponding numeric value

ret_val(X)Similar to *set_val* but works on the data stored in X **Parameters****X** – the data to be converted to numbers**Returns**translation of X **set_val()**

Generates the numeric translation of the given data.

Returns

list of values

class `scheduling.synthdat.field(fld)`

A generic class to handle simple constraints on columns. Accepts only one parameter which refers to a column in the DataFrame.

USE CASES

Use cases show how to apply the general scheduling API to a particular domain. Their data assumptions and dependencies are optional and are not requirements for using the core library.

3.1 PCB call-center workload planning

This is a historical case study applying the scheduling toolkit to aggregated call-center workload data. It is provided as an example of domain adaptation, not as a default package workflow or production staffing recommendation.

The study models emergency and non-emergency calls, generates synthetic 30-minute workloads from aggregated observations, and compares simple first-come-first-served, preemptive, and optimization-based schedules. Results depend on assumptions about call durations and arrivals; review the method and limitations before adapting it.

3.1.1 Install the optional dependencies

```
python -m pip install "scheduling-toolkit[pcb]"
```

The package does not include the underlying Perimeter or 911 call records. Supply a compatible export yourself and ensure you have permission to use it. The script at `examples/pcb_workload.py` demonstrates the data flow.

PCB workload-planning case study

This section documents a historical application of the general scheduling library. It is an illustrative use case, not a description of the package's default purpose or operational guidance. The study used domain-specific assumptions and data supplied by its authors.

The Police Communication Branch (PCB) handled emergency and police non-emergency calls. The study considered the workload transferred to police evaluation staff after an initial assessment by 911 operators.

The workflow considered three stages:

1. 911 primary call center
2. Evaluation unit
3. Dispatch unit

The 911 primary operators take all the incoming calls and after an initial assessment transfer the calls to the appropriate unit for further response. These units include Police Evaluation, EMS, Fire Department, and others.

The evaluation unit processes emergency and non-emergency calls. This case study asked how staffing levels and break schedules might affect the workload of evaluators at different times of day.

The original analysis used historical aggregate data to estimate staffing requirements for 30-minute intervals. The estimates are specific to that dataset and its assumptions.

Objective

Estimate evaluator staffing levels that can handle the modeled workload across the day.

The model considered scheduled breaks as well as workload, using 30-minute time intervals.

Note

Average speed of answer (ASA) and resource utilization can conflict: adding staff may reduce modeled waiting time while lowering utilization. The analysis therefore considered both measures rather than treating either as an unconditional objective.

Limitations of Modeling

It is worth emphasizing that modeling processes that involve a lot of different parameters is a highly complex task, let alone the processes that involve human-based decision making along the way. As [George Box](#) once said “*All models are wrong, but some are useful*”. Once this is clarified and expectations are adjusted, modelers can relax and begin to approximate the phenomenon they are given for investigation.

There are standard models that are proven to work very well for certain problems, again it must be clear that since we are always trying to increase the *accuracy* of models, there is no unique solution model for a given problem. Moreover, looking at a problem from various angles is always a good practice and when different approaches approximately confirm each other, one could trust either as a reliable model based on their needs. Also, different approaches may reveal different aspects of the problem that were hidden from the scope of the other existing solutions.

Scheduling fundamentals

In a scheduling problem we have to fulfill a set of tasks using a number of resources under certain constraints such as restrictions on the task completion times, priorities between task (one task cannot start until another one is finished), etc. The goal is to optimize some criterion, e.g, to minimize the total processing time, which is the completion time of the last task or to maximize the number of processed tasks.

Scheduling approaches

There are various ways to come up with a scheduling method that affects the overall performance of the evaluation unit. Each method assumes certain routines on the evaluation side in terms of handling the calls. In what follows, we describe a few methods and later on we investigate the implication of each one on the average speed of answering (emergency) calls.

First Come, First Serve with priority

Next, we consider a more primitive method for scheduling calls known as **First Come, First Serve with priority**. In this method, agents try to handle all calls in the order they are presented. The only assumption is for when an agent becomes available, the emergency calls in the queue are given higher priority and offered prior to the non-emergency ones. Calls are not interrupted while the agent is busy by one (either emergency or non-emergency). It must be clear that this method will result in an increase in ASA for emergency calls but a possible decrease in ASA for non-emergency ones.

Preemptive Scheduling Model

In this scheduling model, calls will be answered as they come in based on the availability of resources with preemption. Meaning that if a call with higher priority is presented, then one of the evaluators who is busy with a lower priority call will put the current call on hold, answers the high priority one and then resumes the call on hold. When a call with high priority is presented but all evaluators are busy, the model waits for a certain time (2 seconds by default) and

then performs preemption, if no one is available by then. The model considers evaluators who don't have another call on hold already. There is no limit on the number of possible priority levels, the model is capable of handling multiple different priority levels as well.

There are various algorithms handling this particular type of scheduling model including one based on the Mixed Integer Linear Programming, one based on Evolutionary Optimization Algorithms, and Agent Based models. The current implementation follows the *Agent Base* modeling technique.

Absolute Optimal (Deterministic)

Let us formulate a very general scheduling problem which subsumes as special cases a great deal of problems studied in the literature. We are given k tasks $T = \{T_1, \dots, T_k\}$ to be processed via l resources $R = \{R_1, \dots, R_l\}$. Consider the following parameters and variables:

- p_{jm} denotes the processing time of the task T_j by the resource R_m .
- r_j and d_j are the release date/time and due date/time of the task j , respectively.
- x_{jm} (binary) determines if the task j is assigned to the resource m : $x_{jm} = 1$ if yes, and $x_{jm} = 0$ otherwise.
- y_{ij} (binary), sequencing variables: $y_{ij} = 1$ if the tasks i and j are assigned to the same resource and the task i precedes the task j , with $y_{ij} = 0$ otherwise.
- s_j and e_j denote the start and end times of task T_j .

$$\left\{ \begin{array}{ll} \min & \sum_j (e_j - d_j) \\ \text{subject to} & \sum_m x_{jm} = 1 \quad j = 1, \dots, k \quad \dagger \\ & \sum_j p_{jm} x_{jm} \leq \max_j d_j - \min_j r_j \quad m = 1, \dots, l \quad \ddagger \\ & s_j + \sum_m p_{jm} x_{jm} - e_j = 0 \quad j = 1, \dots, k \quad \clubsuit \\ & e_i - s_j + U y_{ij} \leq U \quad i \neq j \in \{1, \dots, k\} \quad \star \\ & r_j \leq s_j \quad j = 1, \dots, k \quad \heartsuit \\ & e_j \leq d_j \quad j = 1, \dots, k \quad \spadesuit \\ & y_{ij} + y_{ji} \leq 1 \quad i < j \in \{1, \dots, k\} \quad \diamond_1 \\ & x_{im} + x_{jm} - y_{ij} - y_{ji} \leq 1 \quad i < j \in \{1, \dots, k\}, m = 1, \dots, l \quad \diamond_2 \\ & x_{in} + x_{jm} + y_{ij} + y_{ji} \leq 2 \quad m \neq n, i < j \quad \diamond_3 \end{array} \right.$$

Here U is a big value to insure the task j will be processed, e.g. $U \geq \max_j d_j - \min_j r_j$, for $m = 1, \dots, l$. It is clear that we can induce customary gaps of block certain time windows by introducing appropriate constraints to the above program.

Associating appropriate weight to different call categories, we can replace the objective function with

$$\sum_{j,m} w_{jm} (s_j - r_j).$$

This will give priority to calls with heavier weights and lower priority to other calls.

Here is a brief description of roles of each set constraints:

- $\dagger$: insures that each job is allocated to exactly one machine
- $\ddagger$: the processing time of each task on all machines does not exceed a threshold
- $\clubsuit$: the start time and end time associated to a task on a certain machine matches the assumed duration of the task
- $\star$: insures each machine finishes its jobs within a desired time interval
- $\heartsuit$: no job begins before it becomes available
- $\spadesuit$: jobs are done before their due

- $\diamond_1$: tracks the order of tasks
- $\diamond_2$: guarantees tasks on the same machine do not overlap
- $\diamond_3$: ordered pair of jobs are not allocated to multiple machines at the same time

Data and method

This use-case note describes how one historical study transformed aggregated Perimeter workload data into synthetic scheduling scenarios. It is not a required input format or workflow for the general-purpose scheduling API.

The evaluation unit data is provided by Perimeter in 30-minutes aggregate format. Hence, determining the exact workload of each agent is not a straight forward task. Among the information in the Perimeter dataset, total duration that agents spent on calls t_{duration} , maximum call duration max_duration , number of completed calls n_{complete} , and number of offered and presented calls n_{offer} , $n_{\text{presented}}$ within the 30 minutes period are present.

A very basic solution to simulate the calls for each period is to take the average duration of calls $\frac{t_{\text{duration}}}{n_{\text{complete}}}$ as the base value for the calls and proceed from there. The following example shows why this approach is not suitable.

Example:

Imagine a scenario where 4 high priority calls are coming through starting at 9, 10, 11, and 15 minutes from the start of the 30-minutes time interval with corresponding durations 18, 12, 15, and 15 minutes, respectively. This adds up to 60 minutes workload.

Also, 10 low priority and rather short calls (3 minutes each) are received by the evaluation unit, which adds up to 30 minutes. Therefore the total workload for this time interval is 90 minutes. The average duration of each call is about 6.42 minutes which can be fit within the schedule for 3 evaluators. While in fact the long calls are overlapping which makes it impossible to fit all the above calls in the given time interval. Fig. 3.1 proposes an optimal schedule that handles all calls given that 4 evaluators are available.

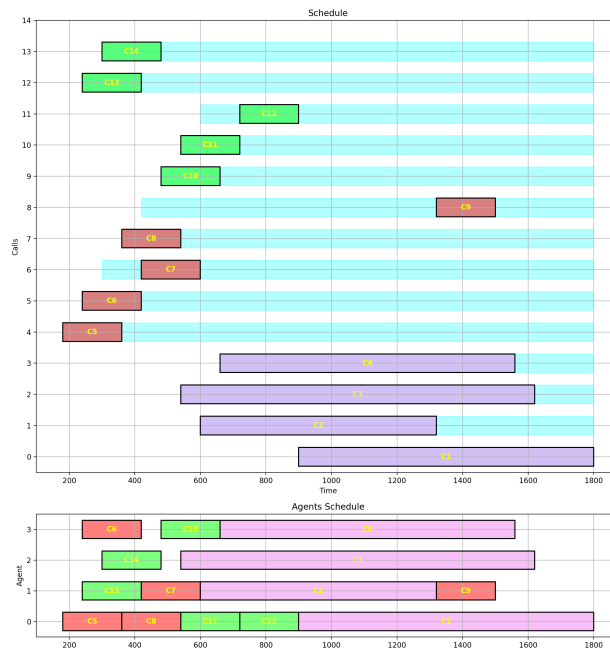


Fig. 3.1: Sample Optimal Schedule

This suggests that using the average duration of calls for analysis is not a wise decision. At the same time, the duration distribution of calls is unknown based on the existing Perimeter data.

Note

To resolve the above issue, let's assume that the duration of calls are following a normal distribution $\mathcal{N}(\mu, \sigma)$. The shift of this distribution is $\mu = \frac{t_{\text{duration}}}{n_{\text{complete}}}$, while the standard deviation σ is unknown. To estimate σ one can choose either of the following solutions:

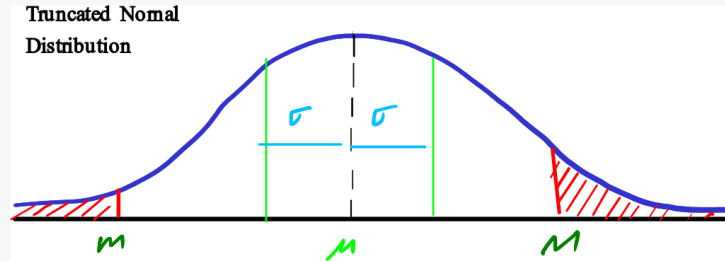


Fig. 3.2: Truncation of a Normal distribution

- Assume that there is only one call with the `max_duration` and $N = n_{\text{complete}}$. Then the probability of having such long calls is $\frac{1}{N}$. Solving the following equation for σ provides an estimate for σ .

$$\int_{\text{max_duration}}^{\infty} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} dx = \frac{\sigma\sqrt{2\pi}}{N}$$

- Assume that the minimum duration is $m \geq 0$ and the maximum duration is $M = \text{max_duration}$. The joint cumulative distribution function for the minimum m and maximum M for a sample of N from a Gaussian distribution with mean μ and standard deviation σ is

$$\Phi\left(\frac{M-\mu}{\sigma}\right)^N - \left[\Phi\left(\frac{M-\mu}{\sigma}\right) - \Phi\left(\frac{m-\mu}{\sigma}\right)\right]^N$$

where Φ is the standard Gaussian CDF. Differentiation with respect to m and M gives the joint probability density function

$$\begin{aligned} f(m, M; \mu, \sigma) &= \\ &N(N-1) \left[\Phi\left(\frac{M-\mu}{\sigma}\right) - \Phi\left(\frac{m-\mu}{\sigma}\right) \right]^{(N-2)} \\ &\cdot \phi\left(\frac{M-\mu}{\sigma}\right) \cdot \phi\left(\frac{m-\mu}{\sigma}\right) \cdot \frac{1}{\sigma^2} \end{aligned}$$

where ϕ is the standard Gaussian PDF. Taking the log and dropping terms that don't contain parameters gives the log-likelihood function

$$\begin{aligned} \ell(\mu, \sigma; m, M) &= (N-2) \log \left[\Phi\left(\frac{M-\mu}{\sigma}\right) - \Phi\left(\frac{m-\mu}{\sigma}\right) \right] \\ &+ \log \phi\left(\frac{M-\mu}{\sigma}\right) + \log \phi\left(\frac{m-\mu}{\sigma}\right) - 2 \log \sigma \end{aligned}$$

This expression has to be maximized numerically to find an estimate for σ .

The following process explains how all the calls transferred to a line are simulated

1. For a row, take $N = \max(n_offer, n_presented)$, $m = 10$, $M = max_duration$, and $\mu = \frac{t_duration}{n_complete}$
2. Find σ using the above algorithms
3. Draw N number from the truncated distribution $\mathcal{N}_m^M(\mu, \sigma)$ as duration of calls
4. Draw N numbers between 0 and 1800 (30 minutes in seconds), uniformly as the start time of calls
5. Combine the above numbers which simulate a hypothetical schedule according to the existing information.

Volume of incoming calls

To get an approximate idea about the workload throughout the day, let us distinguish between number of *Emergency* and *Non-Emergency* calls arriving at each 30-minutes time slot based on two years of Perimeter data (Fig. 3.3 and Fig. 3.4).

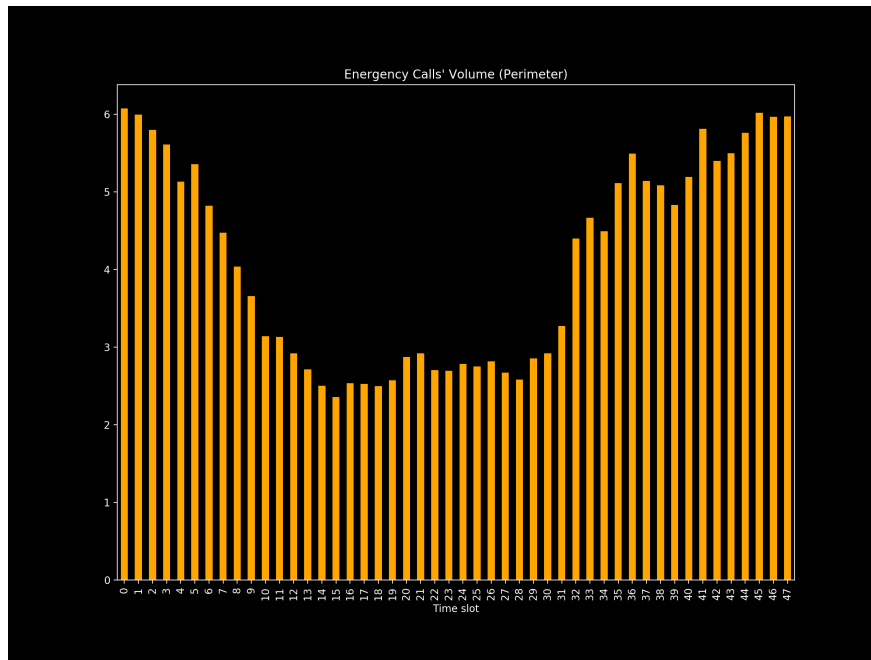


Fig. 3.3: Volume of incoming Emergency calls

A peak at the volume of calls transferred to evaluation from 911 primary Fig. 3.5 confirms that the perimeter data and Telus data align with each other.

To generate the volume of both Emergency and Non-Emergency incoming calls, we are going to assume that the distribution of incoming calls per time slot follows a normal distribution. Then we calculate the parameters of this distribution according to the Perimeter data, plug in those values to simulate a hypothetical day for any given 30-minutes time slot. The following plots (Fig. 3.6 & Fig. 3.7) show the actual distributions extracted from Perimeter data.

These plots resembles collections of **Truncated Normal** distributions which validates the above assumption.

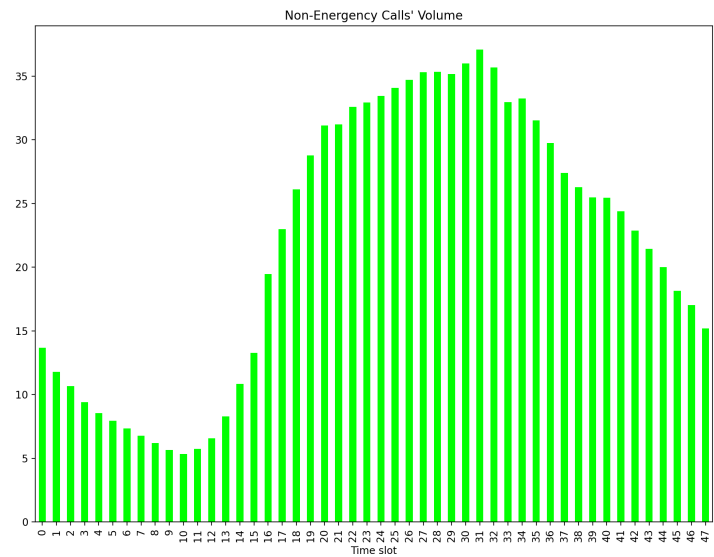


Fig. 3.4: Volume of incoming Non-Emergency calls

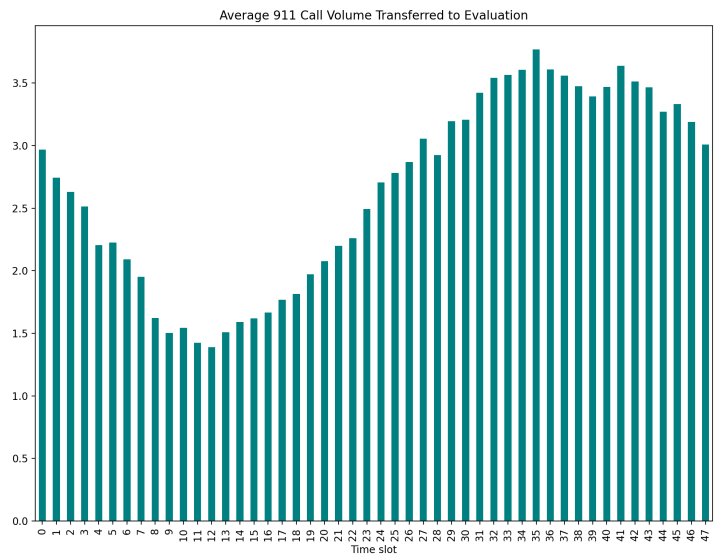


Fig. 3.5: Average volume of transferred calls from 911

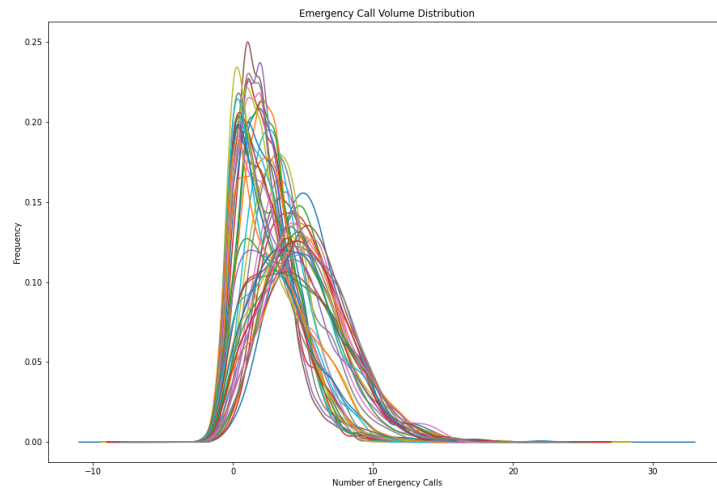


Fig. 3.6: Distribution of incoming Emergency calls

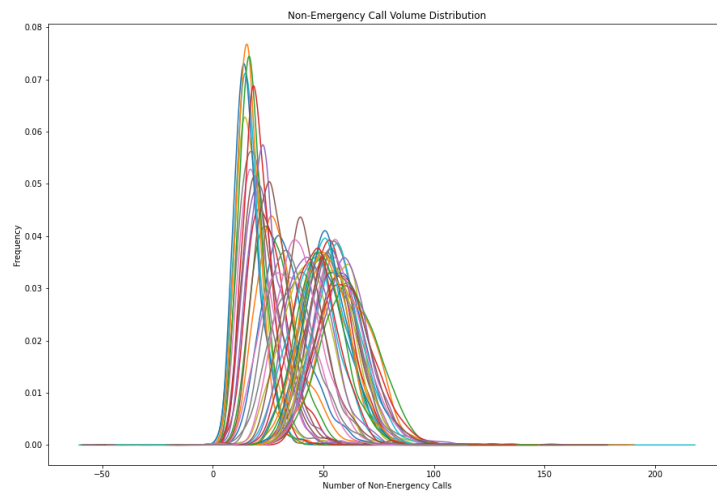


Fig. 3.7: Distribution of incoming Non-Emergency calls

Stochastic Schedule Analysis

Now, we use the above information and processes to generate random calls with different priorities for various time slots. We also assume that each evaluator takes a 2 to 8 minutes **break** per 30 minutes time slot.

To get a more realistic idea about the workloads and performances, we generate 50 random scenarios per time slot for 3 and 4 evaluators. A typical optimal schedule looks like Fig. 3.8.

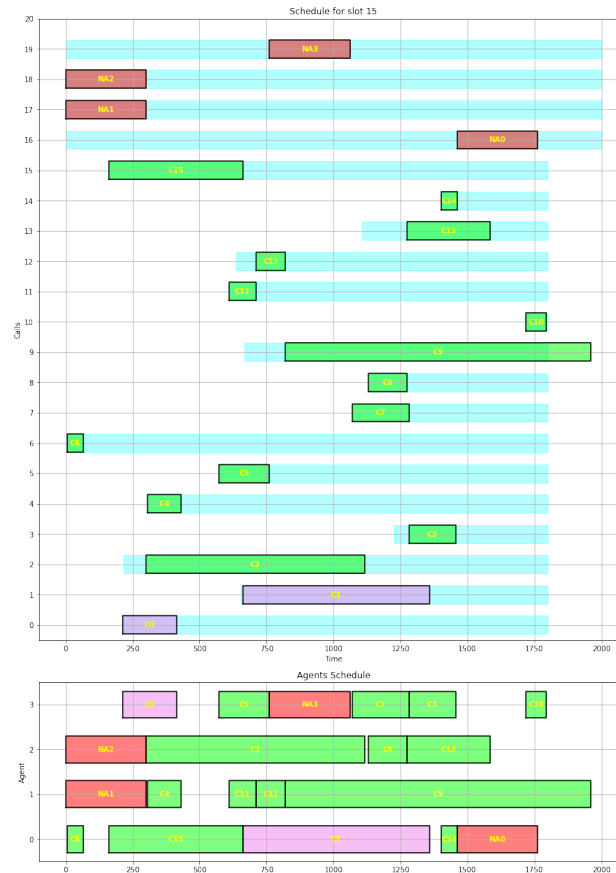


Fig. 3.8: An optimal schedule for 4 evaluators

Preliminary Results on Staffing

The preemptive model was run over the simulated data for all 48 timeslots with different number of evaluators in place. This simulation was repeated 50 times to measure the ASAs, Utilization, and their corresponding 95% confidence intervals. Table 3.1 shows the minimum number of evaluators required to achieve Emergency ASA below 2 seconds, the corresponding Non-emergency ASA, their 95% upper bound, and the average expected utilization.

Table 3.1: Minimum Required Resource

Index	Timeslot	Minimum	ASA	ASA Upper	Non-EM-ASA	Upper Bound	Utilization
0	00:00	8	0.9807	1.8733	8.4709	23.3209	49.70
1	00:30	8	0.5154	0.5663	1.0950	2.2133	43.72
2	01:00	8	0.6467	0.9103	2.0294	4.4187	44.77
3	01:30	5	1.9684	3.6591	15.0067	31.4791	57.80

continues on next page

Table 3.1 – continued from previous page

Index	Timeslot	Minimum	ASA	ASA Upper	Non-EM- ASA	Upper Bound	Utilization
4	02:00	8	0.6461	1.0077	0.6163	0.9394	37.26
5	02:30	5	1.3391	2.7579	2.9601	5.2445	53.33
6	03:00	6	0.3831	0.4748	0.7258	1.1058	38.82
7	03:30	5	1.3210	2.4955	3.2139	6.3450	46.35
8	04:00	5	0.5732	0.7032	1.1280	2.0734	39.54
9	04:30	5	0.7653	1.3828	2.0961	4.4098	40.38
10	05:00	4	0.7523	1.0860	2.3083	4.0776	41.63
11	05:30	4	0.8881	1.3145	7.3983	16.9285	49.96
12	06:00	4	1.9951	4.6979	5.1747	11.5476	45.52
13	06:30	4	1.1180	2.2454	3.0685	5.3090	47.89
14	07:00	5	0.5518	0.6643	2.5639	4.8813	49.89
15	07:30	5	0.5767	0.7251	3.0594	5.3394	48.16
16	08:00	9	0.4962	0.5992	0.4975	0.5318	45.14
17	08:30	8	0.5317	0.6946	3.6832	6.6077	56.70
18	09:00	9	1.1273	2.2669	3.8138	8.1027	56.02
19	09:30	9	0.5828	0.7383	8.1237	15.4273	62.23
20	10:00	10	1.3839	2.6395	13.8913	30.8402	61.99
21	10:30	10	0.5943	0.7261	5.8964	13.8142	60.86
22	11:00	12	0.5230	0.6483	0.4923	0.5156	48.56
23	11:30	10	0.5073	0.6291	4.1672	6.9196	63.48
24	12:00	9	1.5881	3.4937	13.9806	25.3669	71.08
25	12:30	12	0.5949	0.7558	1.7774	3.4218	54.70
26	13:00	12	0.4850	0.6122	2.1081	4.9460	56.79
27	13:30	13	0.6054	0.7224	6.3373	14.6660	56.09
28	14:00	12	0.9037	1.8848	2.9823	6.9839	56.71
29	14:30	12	1.3013	2.9203	7.3702	17.0771	57.41
30	15:00	13	0.5163	0.6589	0.6311	0.9411	51.74
31	15:30	12	0.8237	1.4580	6.8628	13.9997	61.35
32	16:00	14	0.8067	1.4152	3.5954	7.5356	56.08
33	16:30	13	0.8675	1.7568	4.4680	11.1452	54.03
34	17:00	12	1.7370	4.1529	7.8506	18.7565	58.15
35	17:30	11	1.4980	3.2992	19.5509	42.6512	64.28
36	18:00	12	0.9937	1.6129	4.0630	8.6971	55.11
37	18:30	11	0.4735	0.5577	2.1979	4.6018	53.69
38	19:00	11	0.5539	0.6366	3.3152	9.0017	49.84
39	19:30	10	0.5288	0.6070	2.8375	5.5163	54.01
40	20:00	11	1.7888	4.2986	4.9194	12.7403	53.02
41	20:30	10	1.3010	2.5268	8.5427	20.5821	53.12
42	21:00	10	0.5259	0.6375	8.4066	19.2426	57.51
43	21:30	10	0.7358	1.2307	3.2608	6.6859	49.57
44	22:00	9	0.5941	0.6983	1.6726	3.6633	54.61
45	22:30	7	1.3112	2.2735	15.5813	29.5775	63.09
46	23:00	8	0.8151	1.1752	2.5183	4.5128	54.50
47	23:30	8	1.6981	3.0315	7.2277	14.3897	54.46

Abandoned Calls

Next we look at the percentage of non-emergency calls that were dropped without being answered by an evaluator. Clearly, if the calls were answered immediately, none of them would be abandoned. Therefore, it is reasonable to assume that non-emergency ASA has a positive impact on the number of abandoned calls and hence proportion of abandoned calls to all presented calls.

The original analysis included a plot of non-emergency ASA against the proportion of abandoned calls; that generated image is not included in this repository. The reported result was a non-linear relationship, with fewer than 30% abandoned calls expected when non-emergency ASA remains below five minutes.

Call-duration assumptions

This appendix records modeling assumptions made in the PCB workload case study. The estimates and interpretations are study-specific and should be validated before being reused with other data.

The Perimeter data provides data on aggregate level which only includes very few pieces of information regarding the duration of individual calls. We made a decision to assume that the duration of calls follow a *truncated normal* distribution which in general is a reasonable assumption. This assumption requires validation which itself requires data.

The Telus 911 dataset provides limited insight about the duration of calls that have been forwarded to to evaluation unit, emergency or non-emergency.

In this chapter, we use the existing information to validate the assumption that a truncated normal distribution is reliable. To this end, we measure the time between call transfer and call disconnect. We identify the calls forwarded to 7809443700 as emergency and those forwarded to 7809443675 as non-emergency. Of course, there bulk of non-emergency calls referred to evaluation are coming through police non-emergency line and hence the accuracy of the outcome is subject to further debates.

For the emergency calls, the [Fig. 3.9](#) resembles a Γ distribution that itself can be roughly approximated as a *truncated normal distribution*.

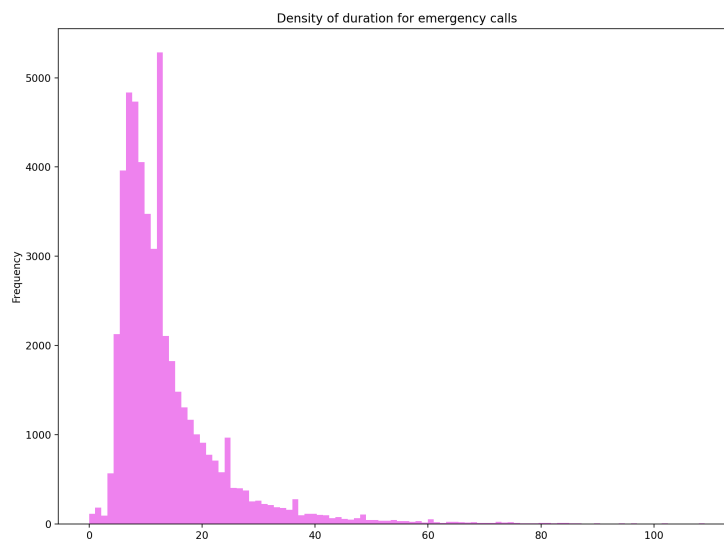


Fig. 3.9: Density of duration of transferred calls to 7809443700

Similarly, the kernel density of the duration of calls transferred to 7809443675 is given in Fig. 3.10. This, again resembles a Γ distribution that we approximate by a truncated normal.

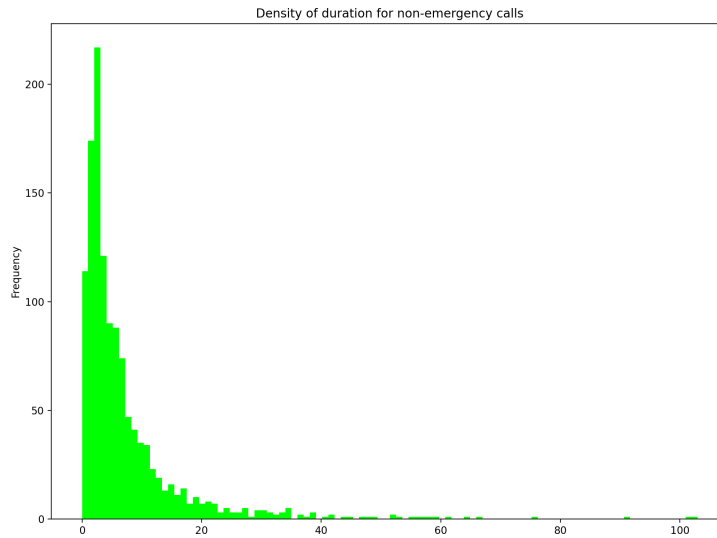


Fig. 3.10: Density of duration of transferred calls to 7809443675

The previously mentioned difference between the mean duration of synthetic samples and the actual mean of Perimeter data can be explained as a result of the rough estimation of the above kernel densities. Note that Fig. 3.9 and Fig. 3.10 show a strong skewness toward the left while a truncated normal looks smoother and skewness is not noticeable.

The density of duration for synthetic non-emergency calls is illustrated in Fig. 3.11

The density of the duration of synthetic emergency call is given in Fig. 3.12.

The comparison among the above density plots shows that our synthetic sample overestimates the duration of evaluation calls. This difference, as explained, is a result of the assumption that the call duration follows a truncated normal distribution.

As a result of this overestimation, the outcome of the proposed analysis is leaning toward the extreme situations where the duration of calls are longer than usual.

Preemptive model results

In the historical case study, the *Preemptive Scheduling Model* was run on all timeslots with various number of evaluators (4 to 14), for 50 different random scenarios according to the Perimeter data.

The original analysis included ASA and utilization plots for each evaluator count. Those generated result images are not included in this repository.

FCFS model results

The historical analysis ran the First Come, First Serve with Priority model with 4 to 11 evaluators in place, 50 random possible scenarios for each time slot to get a 95% confidence interval. Available ASA plots are shown below.

The utilization plots from the original analysis are not included in this repository.

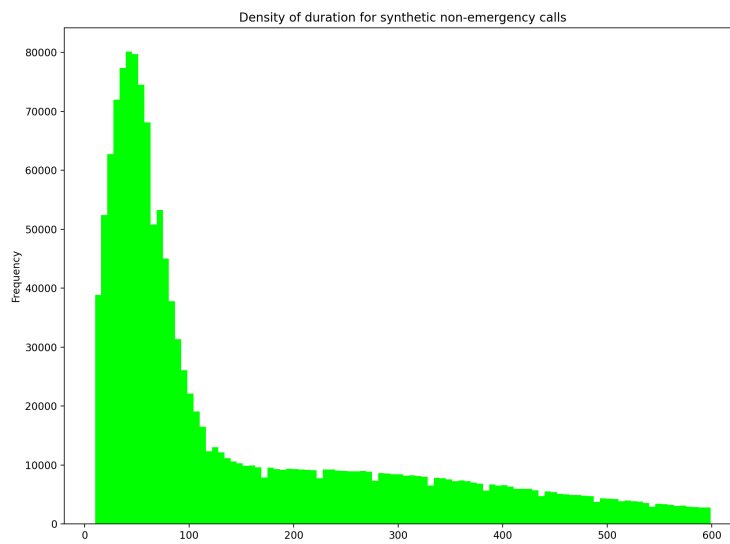


Fig. 3.11: Density of duration of synthetic non-emergency calls

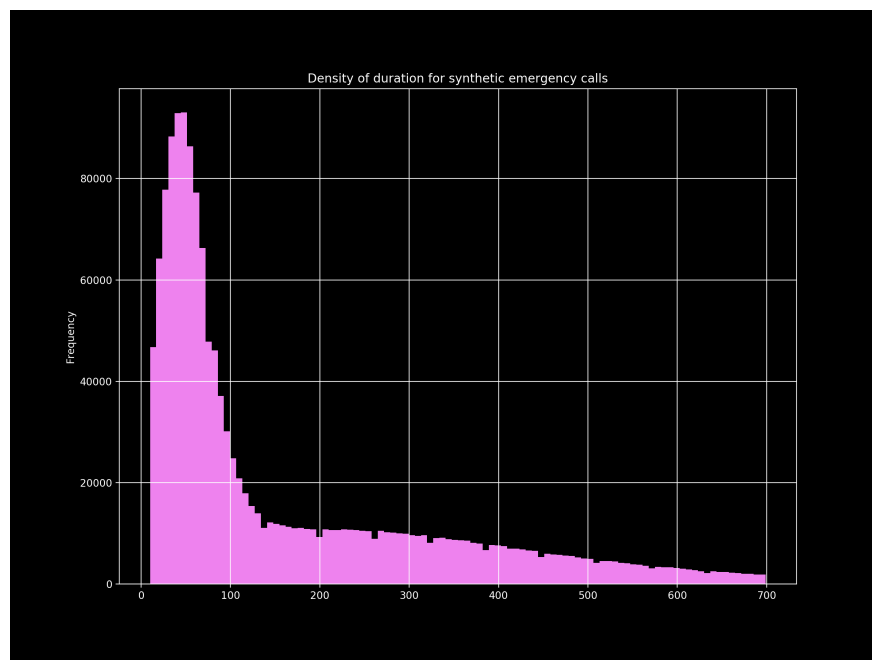
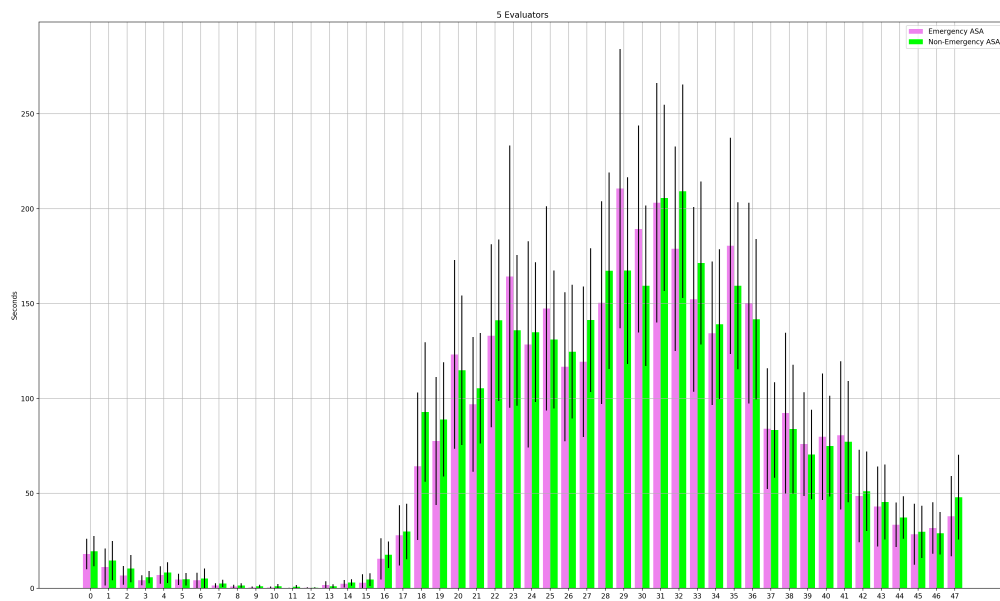
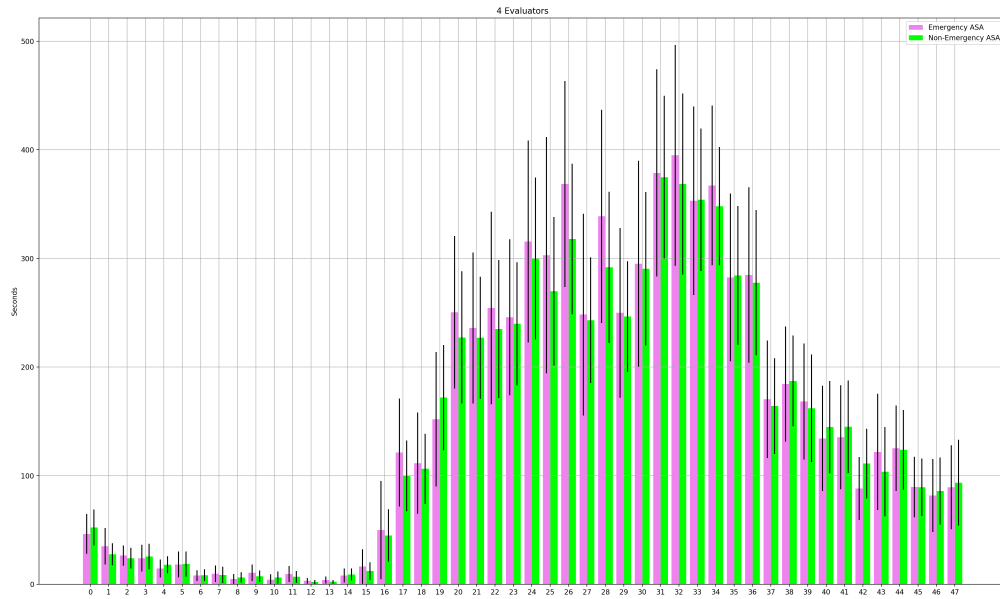
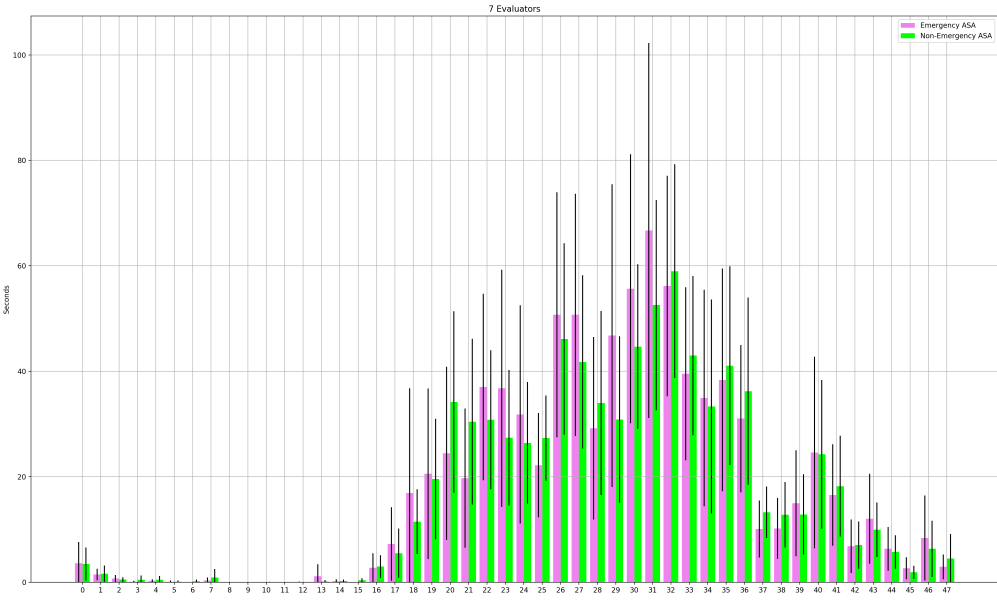
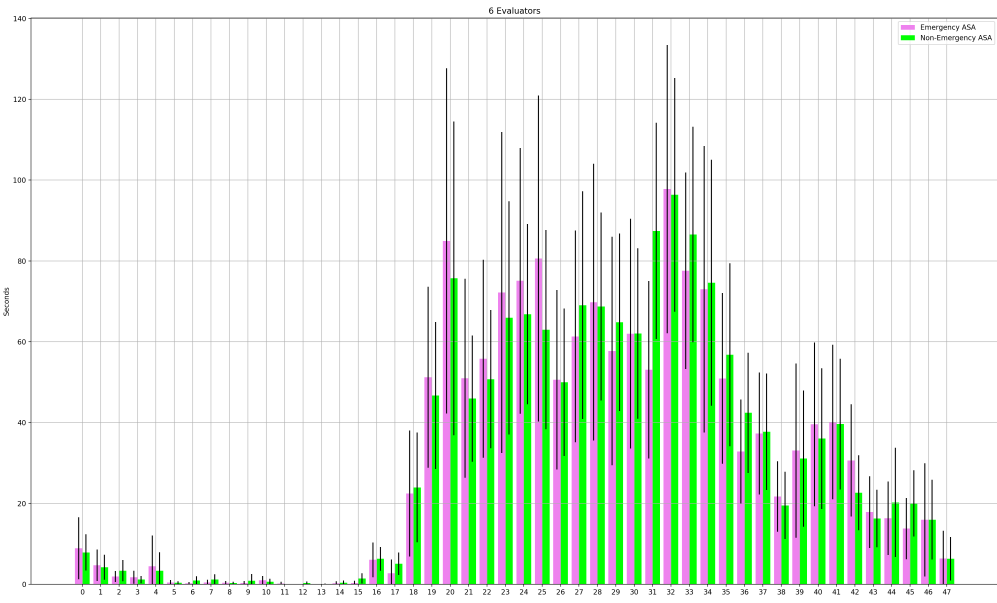
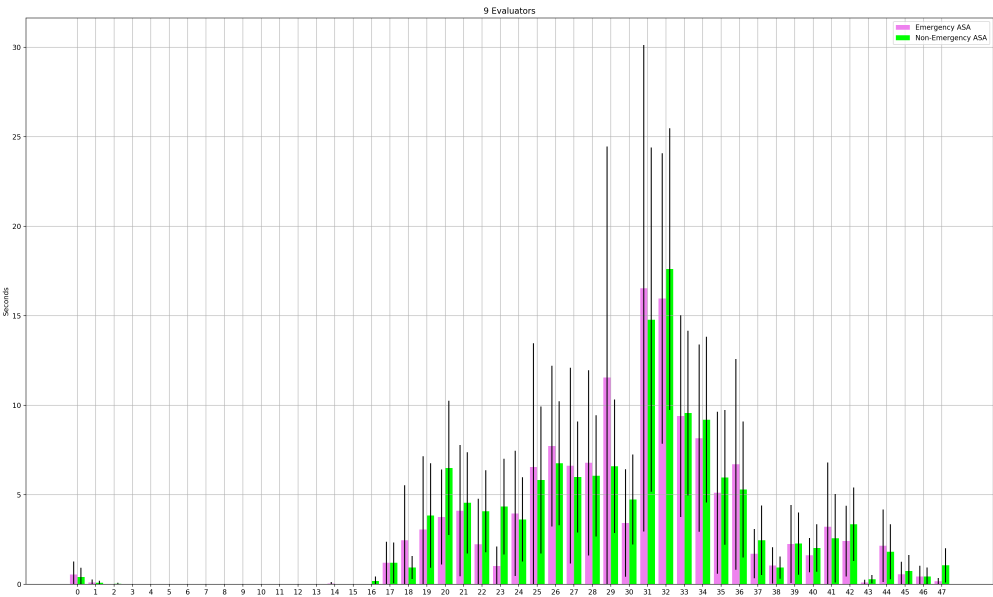
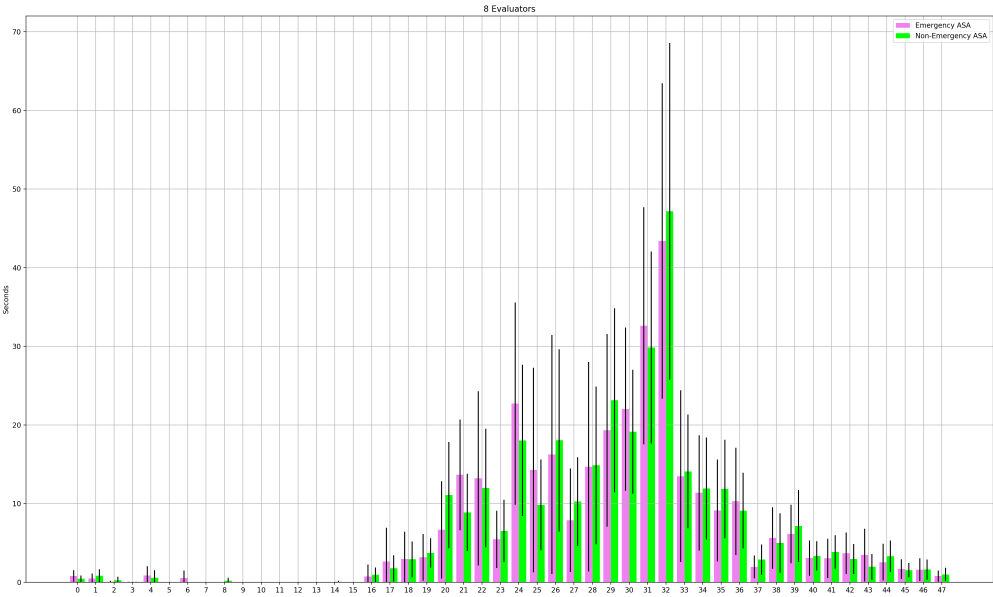
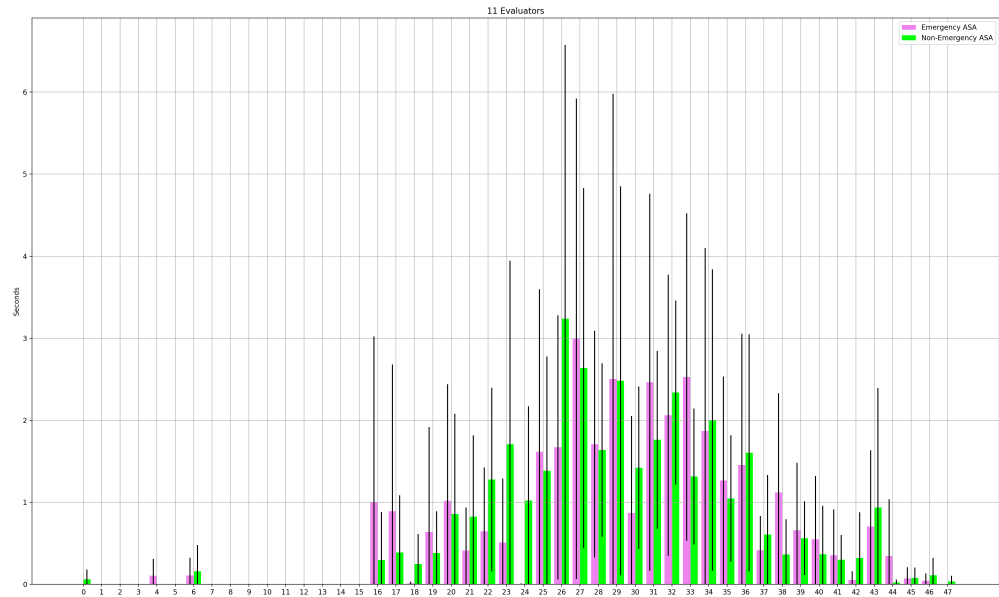
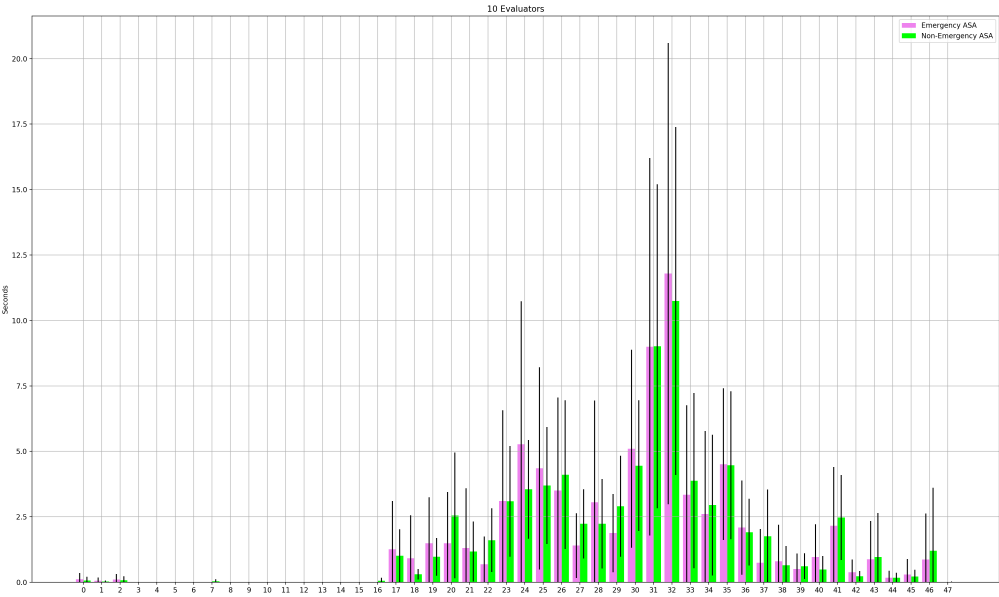


Fig. 3.12: Density of duration of synthetic non-emergency calls









Optimization results

The preliminary PCB analysis showed that with 3 evaluators at work the emergency ASA could go as high as 6.3 seconds (with 95% confidence interval $[0, 15, 5]$). The maximum utilization of evaluators on average is 77.8% (95% confidence interval $[64.9, 90.8]$).

With 4 evaluators, the maximum emergency ASA is 2.75 seconds (95% confidence interval $[0, 6.9]$). The maximum utilization is 65.3% (with 95% confidence interval $[51.9, 78.7]$).

Note

Due to limitation on the processing, we only consider time intervals with low volume of calls (5:00 am to 8:00 am). Moreover, we run the schedule optimization for at most 5 minutes.

The Average Speed of Answering the calls for both emergency and non-emergency are illustrated below with 3 and 4 evaluators and 95% confidence intervals (Fig. 3.13 & Fig. 3.14):

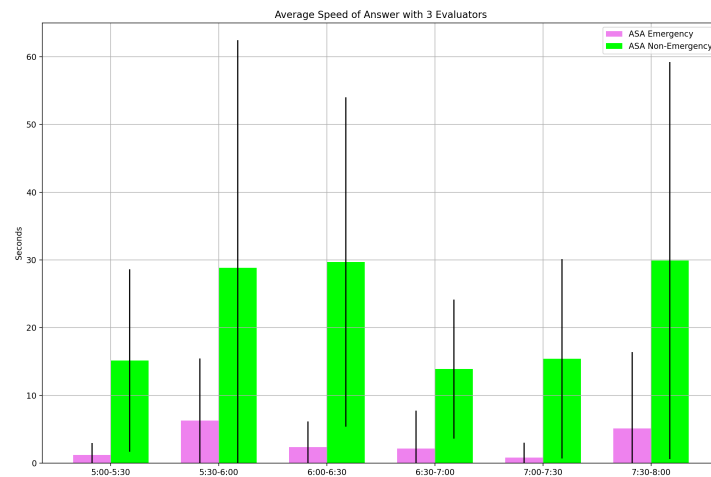


Fig. 3.13: ASA for emergency and non-emergency with 3 evaluators

The amount of time each agent spent on calls on average is Fig. 3.15 illustrated below with lines indicating 95% confidence intervals.

Note

The model was **only** executed for hours between 5:00 am to 8:00 am, where the call volume is minimal, so the a machine with low resources can handle the computational complexity of the model within a reasonable time. Also, the parameters for the call duration distribution is set in a way that usually generates samples that on average are longer than what is being suggested by data. Therefore, the workload is slightly higher than what we may see in the actual Perimeter data.

Note

The model is written in *Python 3.7* following *PEP8* standards. The optimization engine used for this analysis is

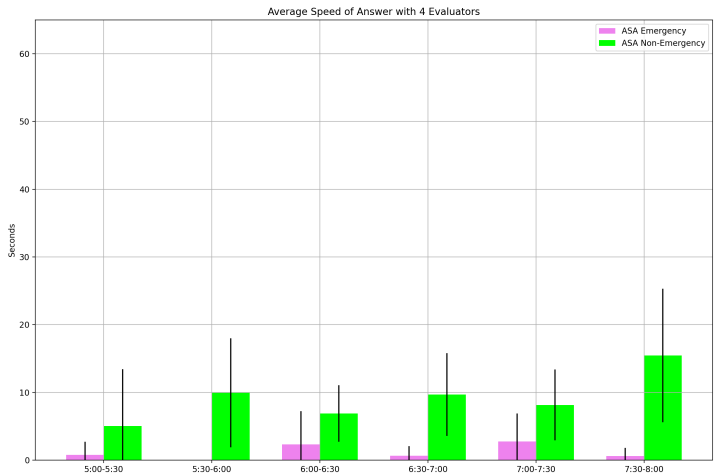


Fig. 3.14: ASA for emergency and non-emergency with 4 evaluators

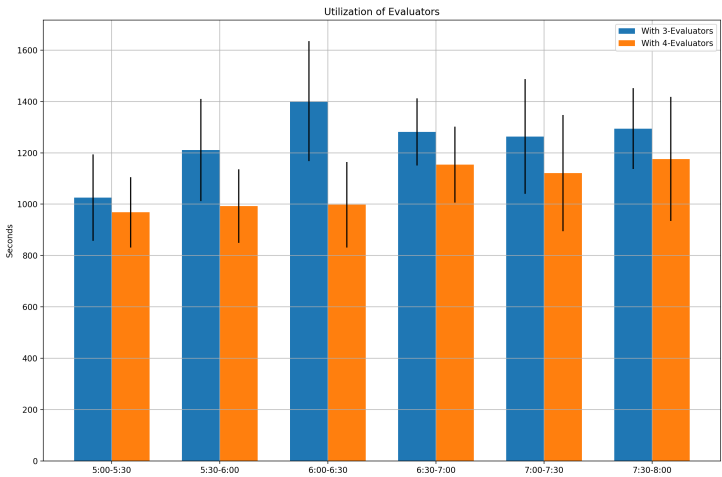


Fig. 3.15: Utilization of Evaluators

IBM's CPLEX, running on UBUNTU 20.04 LTS equipped with 24 GB memory and Intel® Core™ i5-6300U CPU @ 2.40GHz × 4.

PCB use-case API

The PCB helpers are optional and live in `scheduling.use_cases.pcb`. Install their dependencies using the `pcb` extra. The `scheduling.pcb` module is retained only for backwards compatibility.

`Perimeter` accepts a CSV or Excel export with the fields used by the historical analysis. In addition to supplying a compatible input file, set `cache_path` if you want to control where the generated synthetic-call cache is stored.

`Primary911` requires the original call-log export and has a separate `cache_path` parameter for its normalized output. Dataset schemas and preprocessing are specific to this use case.

PCB Specific Module

```
class scheduling.use_cases.pcb.Perimeter(skiprows=1, fname=None, div=1, calc_std=True,  
                                         start_date=None, end_date=None, cache_path=None)
```

Takes a `Perimeter` excel file for pre processing and scheduling

```
evaluation_schedule(time_slot, num_res=4, sto=5, priority=False, dow=None, model='lp',  
                    visuals=False, solver='glpk', executable=None, solver_options=None)
```

Finds the performance measures for a automatically generated optimal schedule

Parameters

- **time_slot** – the index of the time slot
- **num_res** – number of available evaluators
- **sto** – scheduled time off, default 5 minutes
- **priority** – *boolean* determines whether the priority of calls has to be considered for schedule optimization or not
- **dow** – pass
- **model** – pass
- **visuals** – pass

Returns

a dictionary of performance measures

```
generate_call_counts(idx, dow=None)
```

Randomly produces number of emergency and non-emergency calls for a given time slot

Parameters

- **idx** – the index of 30-minutes time slot
- **dow** – pass

Returns

tuple (number of emergency calls, number of non-emergency calls)

```
generate_synth()
```

Generates a `DataFrame` with synthetic calls information, including arrival time, duration, and their dn.

Returns

a `DataFrame` including all generated synthetic data

sample(*idx*, *dow=None*)

Randomly samples the synthetic dataset for the given time slot

Parameters

- **idx** – the index of the time slot
- **dow** – the day of week [0, 1, 2, 3, 4, 5, 6]

Returns

DataFrame

stochastic(*time_slot*, *num_res=4*, *iter=10*, *timeout=300*, *ci=0.95*, *dow=None*, *model='lp'*, *visuals=False*, *solver='glpk'*, *executable=None*, *random_state=None*)

Runs the schedule evaluation procedure stochastically

Parameters

- **time_slot** – the index of the time slot
- **num_res** – number of evaluators
- **iter** – number of iterations
- **timeout** – time limit to execute the optimization
- **ci** – confidence level (default 95%)
- **dow** – pass
- **model** – pass
- **visuals** – pass

Returns

dictionary of performance measures with confidence intervals

class `scheduling.use_cases.pcb.Primary911`(*fname=None*, *cache_path='./data/911_primary.csv'*)

The *Primary911* class uses Telus data to simulate calls and measure the average speed of answering 911 calls with different number of operators in place.

sample_schedule(*rng=None*, *num_res=4*, *extra_load=0.1*)

Samples calls from the dataset according to the daily average

Parameters

- **rng** – list of hours; subset of [0, ..., 23]
- **num_res** – number of primary operators
- **extra_load** – added extra workload; default=10%

Returns

schedules for each hour

stochastic(*num_res=4*, *iter=10*, *ci=0.95*)

Runs the First Come, First Serve scheduling method on 911 calls and determines the 99% confidence intervals.

Parameters

- **num_res** – Number of operators
- **iter** – Number of iterations
- **ci** – confidence level

Returns

averages and confidence intervals

`scheduling.use_cases.pcb.conf_int(data, ci=0.95)`

Finds the confidence intervals given the confidence level *ci*

Parameters

- **data** – The array of numbers
- **ci** – confidence level

Returns

a triplet consisting of mean, lower limit and upper limit of the confidence interval.

`scheduling.use_cases.pcb.find_asas(res)`

Returns the average speed of response to each category of tasks based on their weight category

Parameters

res – the suggested schedule

Returns

the average speed of response for each weight

`scheduling.use_cases.pcb.find_std(mx, size, mu=None, mn=10)`

Finds the standard deviation of a normal distribution when minimum, maximum, size of data, and the average are known

Parameters

- **mx** – known maximum of data points
- **size** – number of data points
- **mu** – average of the data
- **mn** – known minimum of data points

Returns

standard deviation of the assumed Normal distribution

`scheduling.use_cases.pcb.utilizations(res)`

Finds the utilization of each resource based on the suggested schedule

Parameters

res – the schedule

Returns

a dictionary of utilization amounts for each resource

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

S

`scheduling.MultiMachine`, 6
`scheduling.synthdat`, 7
`scheduling.Task`, 5
`scheduling.use_cases.pcb`, 32
`scheduling.Visuals`, 6

INDEX

A

`add_prerequisite()` (*scheduling.Task.Tasks* method), 5
`add_task()` (*scheduling.Task.Tasks* method), 5

C

`conf_int()` (in module *scheduling.use_cases.pcb*), 34

E

`evaluation_schedule()` (*scheduling.use_cases.pcb.Perimeter* method), 32

F

`field` (class in *scheduling.synthdat*), 11
`filter()` (*scheduling.synthdat.SynthData* method), 9
`find_asas()` (in module *scheduling.use_cases.pcb*), 34
`find_std()` (in module *scheduling.use_cases.pcb*), 34

G

`gantt()` (*scheduling.Visuals.Visuals* method), 6
`generate()` (*scheduling.synthdat.SynthData* method), 9
`generate_call_counts()` (*scheduling.use_cases.pcb.Perimeter* method), 32
`generate_synth()` (*scheduling.use_cases.pcb.Perimeter* method), 32
`get_val()` (*scheduling.synthdat.SynthBase* static method), 8
`get_val()` (*scheduling.synthdat.SynthBin* method), 8
`get_val()` (*scheduling.synthdat.SynthCat* method), 9
`get_val()` (*scheduling.synthdat.SynthDate* method), 10
`get_val()` (*scheduling.synthdat.SynthInt* method), 11
`get_val()` (*scheduling.synthdat.SynthReal* method), 11

M

module
 scheduling.MultiMachine, 6
 scheduling.synthdat, 6
 scheduling.Task, 5
 scheduling.use_cases.pcb, 32
 scheduling.Visuals, 6

MultiMachineMILP (class in *scheduling.MultiMachine*), 6

P

Perimeter (class in *scheduling.use_cases.pcb*), 32
Primary911 (class in *scheduling.use_cases.pcb*), 33

R

`ret_val()` (*scheduling.synthdat.SynthBase* method), 8
`ret_val()` (*scheduling.synthdat.SynthBin* method), 8
`ret_val()` (*scheduling.synthdat.SynthCat* method), 9
`ret_val()` (*scheduling.synthdat.SynthDate* method), 10
`ret_val()` (*scheduling.synthdat.SynthInt* method), 11
`ret_val()` (*scheduling.synthdat.SynthReal* method), 11

S

`sample()` (*scheduling.synthdat.SynthData* method), 10
`sample()` (*scheduling.use_cases.pcb.Perimeter* method), 32
`sample_schedule()` (*scheduling.use_cases.pcb.Primary911* method), 33
`schedule_unavailable()` (*scheduling.Task.Tasks* method), 6
scheduling.MultiMachine
 module, 6
scheduling.synthdat
 module, 6
scheduling.Task
 module, 5
scheduling.use_cases.pcb
 module, 32
scheduling.Visuals
 module, 6
`set_type()` (*scheduling.synthdat.SynthData* method), 10
`set_val()` (*scheduling.synthdat.SynthBase* method), 8
`set_val()` (*scheduling.synthdat.SynthBin* method), 8
`set_val()` (*scheduling.synthdat.SynthCat* method), 9
`set_val()` (*scheduling.synthdat.SynthDate* method), 11
`set_val()` (*scheduling.synthdat.SynthInt* method), 11
`set_val()` (*scheduling.synthdat.SynthReal* method), 11

`stochastic()` (*scheduling.use_cases.pcb.Perimeter method*), 33

`stochastic()` (*scheduling.use_cases.pcb.Primary911 method*), 33

`SynthBase` (*class in scheduling.synthdat*), 8

`SynthBin` (*class in scheduling.synthdat*), 8

`SynthCat` (*class in scheduling.synthdat*), 9

`SynthData` (*class in scheduling.synthdat*), 9

`SynthDate` (*class in scheduling.synthdat*), 10

`SynthInt` (*class in scheduling.synthdat*), 11

`SynthReal` (*class in scheduling.synthdat*), 11

T

`Tasks` (*class in scheduling.Task*), 5

`transform()` (*scheduling.synthdat.SynthData method*), 10

U

`utilizations()` (*in module scheduling.use_cases.pcb*), 34

V

`Visuals` (*class in scheduling.Visuals*), 6

W

`where()` (*scheduling.synthdat.SynthData method*), 10